

FEWLLM Data Agent

KDD Cup 2026 DataAgent-Bench — 课程项目

马瑞鑫 林译鹏 肖尧

信息枢纽
香港科技大学（广州）

2026 年春季



香港科技大學（廣州）
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

① 问题与系统架构

任务定义、评分函数、四层设计

② Agent 主循环与路由

TaskCompiler、React Harness (协议/上下文/Self-verify)、Operator cascade 流水线

③ 风险闸门 (核心创新)

Cheap guard、惰性升级、可审计 trace

④ 支撑组件

RAG 工具、多 Agent 编排、self-consistency

⑤ 演示与结论

任务形式与评分函数

输入: `task.json` + 异构上下文目录
(CSV、SQLite、JSON、Markdown 规则文档、
自然语言记录、图像)

输出: `prediction.csv`, 由
列内容签名匹配评分

- 列名忽略; 行顺序忽略
- 多列匹配走二部图最大匹配
- 归一化: 数值 $\rightarrow 10^{-2}$, 字符串
`strip+lower`

$$\text{score} = \max\left(0, \text{recall} - \lambda \cdot \frac{\text{extra_cols}}{\text{pred_cols}}\right), \quad \lambda = 0.5$$

设计含义

- 列要少而准: 每多一列冗余都会扣分
- 保留粒度: `first_name+last_name` 优于
合并的 `name`
- 类型一致: $163109.0 \neq 163109$

设计目标：失败模式与设计原则

六类可观察失败模式

- 答案缺乏数据支撑：依文件预览或先验直接作答
→ 形状对、内容错。
- 字段语义绑定错误：将概念词当列名/过滤值；目标实为字段值。
- 工具协议不稳定：自由文本 JSON 易被转义/括号问题打断。
- 错误信息可利用性不足：原始异常不足以指导自我纠正。
- 程序静态错误：codegen 引用不存在的文件、表、列。
- 任务级边界控制：验证、修复、路径切换可耗尽时间。

四条设计原则

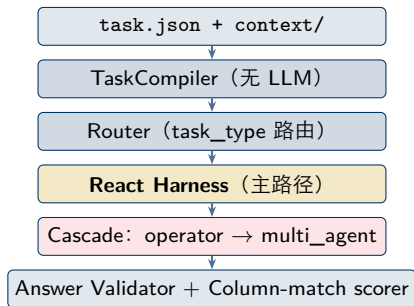
- 先扫描，再推理：任何模型推理前先做无 LLM 的上下文画像。
- 工具行为显式化：所有数据访问与计算经统一工具完成，每步动作写入 `trace.json`。
- 本地检查优先：结构、静态、风险评估、可机械修复的错误尽量由确定性规则处理；必要时再升级到 LLM。
- 路径失败可复盘：路由选择、失败原因、兜底路径与最终答案均结构化写入 `trace`。

核心权衡

模型负责“规划与生成”，确定性模块负责“约束与验证”。这条边界使主路径在不牺牲可解释性的前提下保持稳定。

端到端架构与上下文进入方式

流水线



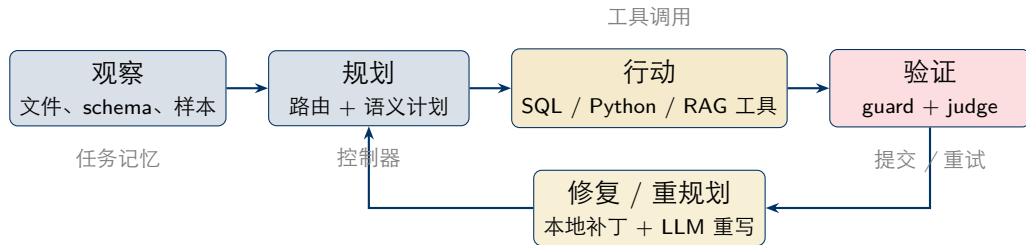
不同阶段以不同粒度进入模型

阶段	送入模型的内容
React 预规划	文件路径 / 类型 / 大小 (最多 25 项)
React 主循环	仅初始问题; 文件内容须经工具读取
Multi-agent planner	文件清单 / schema / 行数 (不含数据行)
Operator codegen	表格样本、JSON 摘要、文档相关章节、RAG top-K 片段

- 预规划阶段不读数据内容; codegen 阶段仅接收预算筛选后的切片。
- 真正计算必须由 `execute_python / execute_context_sql` 在原始上下文中完成。

为什么使用 Agent 框架：控制器、工具、记忆与验证

裸 **ReAct** 不足以应对该任务：简单题在重复工具调用中浪费 token；复杂题在中间观察间漂移；最终表格需要严格的形状控制与语义验证。



设计原则

Agent 维护一份任务 *trace*，自主选择工具，观察工具输出，仅当验证模块接受答案时停止。
简单任务依然只支付最小 *LLM* 成本。

相对 LangChain / LangGraph 的定位

LangChain 词汇

- Agent = 模型 + 工具 + 迭代循环
- 运行时图 = 模型节点、工具节点、提交/重试边
- 中间件控制上下文、工具访问与护栏
- LangSmith 风格 trace 用于调试

本系统对应实现

- ModelAdapter = 提供方抽象
- ToolRegistry = 强类型工具目录
- ReActAgent = 模型—工具循环
- Orchestrator = planner-specialist-synthesizer DAG
- trace.json = 记忆 + 审计

定位

本项目使用一个轻量的、LangGraph 风格的运行时，而非直接依赖 LangChain。该架构与 LangChain 适配器兼容：将 ToolRegistry 包装为 LangChain 工具，复用同一套路由器、验证器与答案校验器即可。

Agent 控制器：按任务画像分派

TaskCompiler (不调用 LLM):

- 逐文件扫描 → SourceCapability
列名、类型、基数、样本
- 任务级聚合 → ExecutionProfile
 - context_size, source_shape
 - verifiability, operation_complexity
 - semantic_rule_required
- **统一事实快照** ——由 prompt、静态检查、guard、judge 共享

路由规则 (确定性):

- 全部 7 种 task_type 统一映射至 **react_harness** 作为首发路径
- 失败回退依据 failure_type 选择:
 - retrieval_empty → RAG-enabled operator
 - semantic_consistency_failed → tool_first_mixed
 - syntax / static / exec_error → 同类 operator
 - zero_rows → 不换路径, 由 local repair 处理
- 单题硬预算: 仅 max_seconds=600, LLM 与工具调用次数不设上限

Cascade 链:

react_harness → tool_first_mixed → extreme → fallback_multi_agent

至多额外尝试 1 次

React Harness 主路径（一）：协议、上下文与知识引导

1. Native tool calling 与文本回退

- ToolSpec 同源维护 prompt 描述与 tools-API JSON Schema，避免协议描述漂移
- 模型经 OpenAI function-call 协议调用工具
- 服务端返回纯文本时退化为 JSON 解析，含三层容错：fence 剥离 / 控制字符转义 / 未闭合括号补齐
- 模型一次返回多个 tool_call 时只采纳第一个，且不记录被丢弃的，避免下一轮 orphan tool message 触发 API 拒绝

2. 工具结果缓存（per-task）

- 缓存键 = (action, sorted(action_input)), 处理参数次序差异
- 仅缓存 9 类只读工具；execute_python 与 answer 不缓存
- 缓存命中永不触发终止，避免重复触发 answer 终结

4. 轻量 pre-loop planner（可选）

- 单次 LLM 调用；仅基于文件清单（最多 25 项）；不读取内容
- Plan 注入为 user 消息前缀，每轮重构 messages 时持续可见
- Planner prompt 强约束第二步必须读 knowledge / rule / glossary 文档
- Plan 解析或调用失败时优雅降级；skip_for_easy 跳过简单题以省一次调用

5. System Prompt 强约束（schema-link first, then code）

- **Knowledge file 不可议价**：knowledge.md / rule / glossary 为单一事实源；先验知识与之冲突时一律以文档为准；要求语义化应用规则而非照抄公式
- **样本行使用约束**：read_csv 等预览行仅可推断 dtype / 格式 / join key，禁止据此输出最终答案

React Harness 主路径（二）：错误归类与 self-verify 闭环

6. 确定性 retry hints（无 LLM）

- 错误模式按工具分类：Python（10 类，覆盖 `KeyError`、`ParserError`、`python_timeout` 等）、SQL（4 类）、Path（1 类）、Answer（3 类）
- 错误签名归一化：去 line N / 路径 / 引号，使同类错误能“碰头”
- `ErrorHistory` 跟踪重复次数：相同错误 ≥ 2 次时提示前置 *"Stop retrying — switch tools or read the data first"*
- 即使工具抛异常未返回 result，仍跑一次 hint 归类（覆盖 answer 行宽错误等）

7. Native tool_call orphan 修复

- 工具异常时仍把 action 名 + tool_call_id 记录入 step
- 下一轮 messages 重建时不会出现 orphan tool message，避免 API 因 tool_call_id 不匹配而 400

8. 强制 self-verification（draft $\rightarrow$ verify $\rightarrow$ final）

- `verification_rounds = 1 + use_answer_guard = true`
- 第一次 answer 不立即终止，被改写为 `status = draft_submitted` observation，附 `draft_columns / draft_row_count / remaining_rounds`
- **Cheap Answer Guard**（下一帧详述）输出风险代码 + `top_risk`
- `build_verify_instructions` 把每个风险代码翻译为具体行动指令；附 *Most important: ...* 突出最高风险

9. 预算与未确认的 draft 兜底

- self-verify 阶段触发 `BudgetExceeded` 时，commit 最近 draft 并标记 `budget_exceeded_during_self_verify`
- 循环结束模型未确认时，回退至最近 draft 并标

React Cheap Answer Guard: 7 类确定性风险

位置: React 主路径首次提交 answer 前; 零 LLM 成本; 与 Operator 路径共享 SemanticRiskAssessment 数据结构。

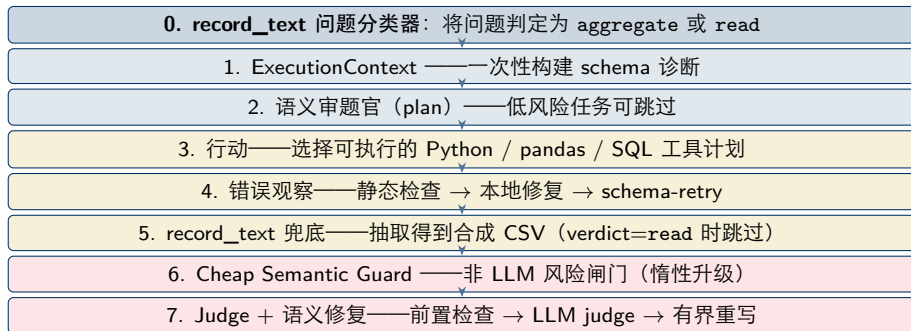
风险代码	触发条件	Severity	Weight
execution_failed_or_missing_answer	answer 调用未产出 AnswerTable	error	1.0
empty_answer_rows	候选答案有列但行数为 0	error	0.75
invalid_answer	validate_answer_table 检查失败	error	1.0
answer_without_computation	提交前未成功调用 execute_python 或 execute_context_sql	error	0.85
knowledge_doc_not_read	上下文存在 knowledge / rule / glossary 文档但 read_doc 从未读取	error	0.8
shape_mismatch_scalar_question	问题语气暗示标量但答案行数 > 1	warning	0.35
shape_mismatch_list_question	问题语气暗示列表但答案为 1 × 1 标量	warning	0.35

升级行为 (与 verification_rounds 协同)

当前配置 verification_rounds=1: 所有任务均走两轮 answer, guard 不再决定是否升级, 而是把发现的风险代码塞进 verify observation, 使模型在重答时有针对性地复核——而非无差别 self-correction。

Cascade 兜底：Operator Executor 流水线（8 决策点）

位置：主路径失败、cascade 进入 `tool_first_mixed` 或 `extreme` 之后；并非主路径流程。



共享证据——`debug_steps`

每段生成的程序都会输出 `schema_inspection`、`used_columns`、`filter_conditions`、`join_keys`、`intermediate_counts`。被 3 个消费方 读取：`cheap guard`、`preflight`、`LLM judge` —— 同一份审计 `trace`，避免下游模块认知漂移。

运行配置、预算控制与 Reasoner Repair

运行配置摘要

维度	配置
Agent 模式	router
主模型	qwen3.5-35b-a3b (DashScope OpenAI-compatible 端点)
单题硬预算	max_seconds = 600; LLM / 工具调用次数不设上限
默认 route	react_harness, max_steps=40, verification_rounds=1, use_answer_guard=true
Cascade	react_harness → tool_first_mixed → extreme → fallback_multi_agent, 至多额外 1 次
RAG (cascade)	use_hybrid=true + DashScope text-embedding-v4 dense + 查询 扩展 (HyDE), 无 reranker
Cross-model verify	关闭

BudgetController: 6 维度

- max_llm_calls (-1 不限)
- max_tool_calls (-1 不限)
- max_seconds = 600 (唯一硬上限)
- max_local_repairs (确定性 fixer 上限)
- max_reasoner_repairs
- max_multiagent_fallbacks

Reasoner Repair (启用, 仅 cascade)

- 触发条件: 当前 route 为 program 类、payload 失败且含已生成程序、needs_reasoner=true 或 large_document_context、预算未耗尽
- React 主路径不属于 program 类, 因此 reasoner repair 仅在 cascade 进入 tool_first_mixed / extreme 之后才会触发
- 失败程序 + stderr/stdout + capabilities 摘要

Schema Grounding + 静态检查 + 本地修复

Schema Grounding

表格类源 *prompt* 提示

- 真实列 TOP-K 候选
- 按源类型：
- **CSV / SQLite** ——完整打分
- **JSON** ——样本 + 字段
- 文本 / 图像——跳过

静态检查

执行前闸门

- Python AST + SQL 扫描
- `no_such_column`、`bad_join_key`、`_x/_y` 后缀
- 在执行之前运行
- 拦截可预见的失败

Local Repair (无 LLM)

确定性补丁器

- 9 类确定性 fixer
- 列名近似替换
- `merge key` 字符串归一
- 缺失 `answer` 自动补齐
- 零行常见过滤修正

运行期安全网

Schema grounding 仅作为 *prompt* 提示。真正的防御来自运行期证据：**schema_inspection** → 静态检查 → **cheap guard** → **LLM judge**。它们读取同一份 `SourceCapability` 与 `trace.json`，审计链路不漂移。

Cheap Semantic Guard: 非 LLM 风险闸门

位置: codegen 已经成功, answer 也通过结构校验 → 是否可以信任该答案?

$$\text{risk_score} = \min(1.0, \sum \text{unique_weights}), \quad \text{escalate} = \text{任一 error 级风险} \vee \text{score} \geq 0.5$$

类别	风险代码	权重	含义
执行	execution_failed / empty_answer_rows	1.0 / 0.75	答案缺失或零行
修复	suspicious_fallback:schema_retry	0.8	依赖 schema retry
Trace	trace_missing:schema_inspection	0.8	缺真实列检查
Trace	zero_intermediate_count	0.8	中间行数降为 0
覆盖	field_coverage_mismatch	0.75	Grounding 候选未被使用
公式	formula_without_trigger	0.75	使用了问题未要求的派生公式

典型识别场景

问题询问“具有特定属性的实体”，但程序使用与属性名仅形似的字段做过滤；schema grounding 已给出更合适的候选字段，却未被代码使用 → field_coverage_mismatch (0.75) → **升级到 judge**，避免直接提交错误答案。

惰性升级 + Analyst 异常恢复

1. codegen 后的闸门

codegen 成功且答案结构合法后，**Cheap Guard** 用 13 类确定性风险打分（无 LLM）。若 $\text{score} < 0.5$ 则 **快速通过**，零额外 LLM 成本；否则升级。

2. 升级后的 LLM 流程

- 审题官 (**Analyst**) (1 次 LLM 调用) —— 输出语义计划
- 执行官 (**Judge**) —— 比较计划与代码
 - pass → 完成
 - fail / low_confidence → 语义修复重写代码
- Judge-Repair 循环最多 3 轮

3. Analyst 异常恢复

stage=analyst_exception → 以 force=True 重试一次；仍失败则构造低置信度的 synthetic plan → **judge 必须运行**。

4. 可观测性——final_gate

trace.json 顶层字段：

- plan+judge —— 主路径
- escalated+judge —— 经过重试 / fallback / 惰性升级后恢复
- cheap_guard_only —— guard 触发但 judge 未运行（需调查）
- bypassed —— 未进入语义验证

5. 成本

- 简单任务：1 次 LLM 调用（仅 codegen）
- 含规则的任务：3-5 次（analyst + codegen + judge ± repair）
- Analyst 异常：最坏额外 +1 重试 +1 judge

核心保证

任何可疑答案都不会被直接提交：一旦 guard 决定升级，judge 必然会运行——即便 analyst 失败。

工具沙箱、知识工具与长文档检索

1. 受控执行环境

- `execute_python` 在独立 `multiprocessing.Process` 中运行；`cwd` 固定为任务 `context/`
- `stdout / stderr` 经 `Queue` 回传；超过 **30 秒** `terminate()/kill()`
- `execute_context_sql` 仅允许只读语句
- 文件读取均切片：`read_csv` 支持 `columns_only/offset`；`read_doc/json` 按 `max_chars` 分页；`grep_doc` 返回命中上下文

2. `consult_knowledge` (可选)

- 仅当 `helper_model.enabled` 且当前 `route` 为 `React` 时注册
- 自动扫描 `knowledge / rule / glossary / definition / schema_notes` 命名的文档
- 由更聚焦的 `helper` 模型解释规则；约束其只能基于给定文档；找不到答案必须返回 `not_specified`
- 把“读规则”与“算数据”解耦：主路径仍由工具完成数据计算

3. **RAG** (`cascade` 才激活)

- 标题感知切片 (1200 字符, 重叠 150)
- BM25 稀疏；可加 `dense embedding`，按 **RRF** 融合
- 查询扩展：改写 + HyDE 假设回答
- 可选 `cross-encoder reranker` (先取 `first_stage_top_n=30`，再 `rerank` 至 `top-K=6`)
- 渐进增强：组件缺失时退化为 BM25

React 主路径上的文档处理

React Harness 不进入 RAG 流水线：文档检索由模型通过 `read_doc / grep_doc / head_doc` 自行完成，RAG 仅当 `cascade`

多 Agent 模式：角色分工与权限隔离

1. Planner

- 职责：将问题分解为子任务 DAG
- 输出：带依赖关系的 JSON plan
- 工具：无；纯推理
- 约束：不能调用 `answer` 或 `report`

2. Specialist (5 类，并行执行)

类型	允许工具
Schema	<code>list_context</code> , <code>read_csv</code> , <code>read_json</code> , <code>read_doc</code> , <code>inspect_sqlite_schema</code>
SQL	<code>list_context</code> , <code>read_doc</code> , <code>inspect_sqlite_schema</code> , <code>execute_context_sql</code>
Python	<code>list_context</code> , <code>read_csv/json/doc</code> , <code>inspect_sqlite</code> , <code>execute_sql</code> , <code>execute_python</code>
Document	<code>list_context</code> , <code>read_doc</code> , <code>read_json</code> , <code>read_csv</code>
Generic	全部工具；完整 ReAct

- 终止动作 = `report`，而非 `answer`
- 同一 `specialist` 内 `read_doc` 至多调用 2 次
- 单 `specialist` 最多 12 个 ReAct 步

3. Synthesizer

- 职责：合并 `specialist` 的 `finding` 为最终答案
- 输入：全部 `report` 内容
- 唯一可调用 `answer` 的角色
- 最多 6 个 ReAct 步
- 合成失败可触发一次 `replan`

4. Orchestrator

- 通过 `ThreadPool` 派发子任务
- 同层 `specialist` 可并行
- 强制执行 DAG 依赖关系
- 第 N 层等待第 $N - 1$ 层完成
- 迭代 `refinement`：至多 1 次重新规划
- 预算上限：`max_multiagent_fallbacks = 1`

关键设计：权限隔离

Specialist 不可直接提交答案，只能 `report` 中间证据。Synthesizer 必须交叉核对所有 `report`，方可最终调用 `answer`。

Self-consistency、预算控制与演示

Self-consistency: 列签名投票

- N 次采样 ($T=0.7$, 不同 seed)
- `target_count` = 列数的众数
- 取频次前 `target_count` 的签名 = 获胜列集合
- 选择最大覆盖、最少冗余的样本
- 直接对齐评分函数
 $\text{recall} - \lambda \cdot \text{extra}/\text{pred}$

预算控制器 (任务级):

- LLM 调用 / 工具调用 / 时间上限
- 本地修复与 `reasoner` 修复上限
- 多 Agent 兜底次数上限
- 上限 = $\min(\text{YAML}、\text{TaskCompiler 推荐})$

Streamlit 演示:

- 子进程实时日志 (ProgressLogger)
- 推理时间线: `schema` → 路由 → `codegen` → `guard` → `judge`
- 最终答案表 + 生成的 Python 代码
- 完整 `trace.json` (可折叠)

```
uv run streamlit run \
  scripts/demo_app.py
```

可审计、可解释、可复现。

Public split 评测结果与失分分析

评测概况

指标	数值
Records	49
Success	47 / 49
Failed	2
Total score	35.600 / 49
Mean score	0.727
Mean recall	0.745
Score = 1.0	34

两条失败任务

- task_344: 单题运行超过 600 秒
- task_396: 语义修复后仍未通过静态检查

部分得分典型

- task_38 = 0.60; task_249 = 0.25; task_379 = 0.75
- 共性: 召回了部分正确列, 但同时输出了冗余列或遗漏标准答案列

四类已知失分原因

- 列投影不够保守: 评分函数对额外列设惩罚; 中间调试列未被剔除。
- 字段语义绑定: 自然语言概念被映射到近似但错误的字段。
- 语义修复后的静态错误: 重写引入新的列名或语法问题。
- 单题超时: 复杂路径耗尽 600 秒上限。

执行轨迹观察

绝大多数任务由 `react_harness` 直接完成; 进入 `cascade` 的任务比例较低, 与系统设计定位一致——主路径承担稳定性与协议约束,

总结与未来工作

已实现的内容：

- 显式工具行动的 Agent 控制器
- 确定性任务画像（无 LLM）
- 按 task_type 路由 + 按 failure_type cascade
- 9 类本地修复规则（无 LLM）
- Cheap Semantic Guard ——非 LLM 风险闸门
- Analyst 异常恢复 + 完整审计 trace
- 工程化 RAG 流水线（BM25+Embedding+RRF）
- 多 Agent DAG + 受限重规划
- 列签名 self-consistency

Public split 评测结果：

49 题平均 0.727, 34/49 满分；总分 35.6/49

未来工作：

- 视觉任务专用执行器
- 基于 trace FP/FN 自动校准 guard 权重
- 列内单值多数投票
- 长 record-text 上更稳定的抽取策略
- 调优后提交 hidden test

谢谢

欢迎提问



代码仓库: github.com/zuelmaruixin/kdd